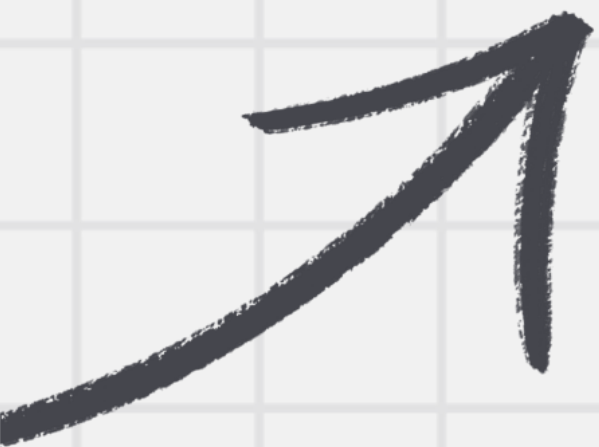
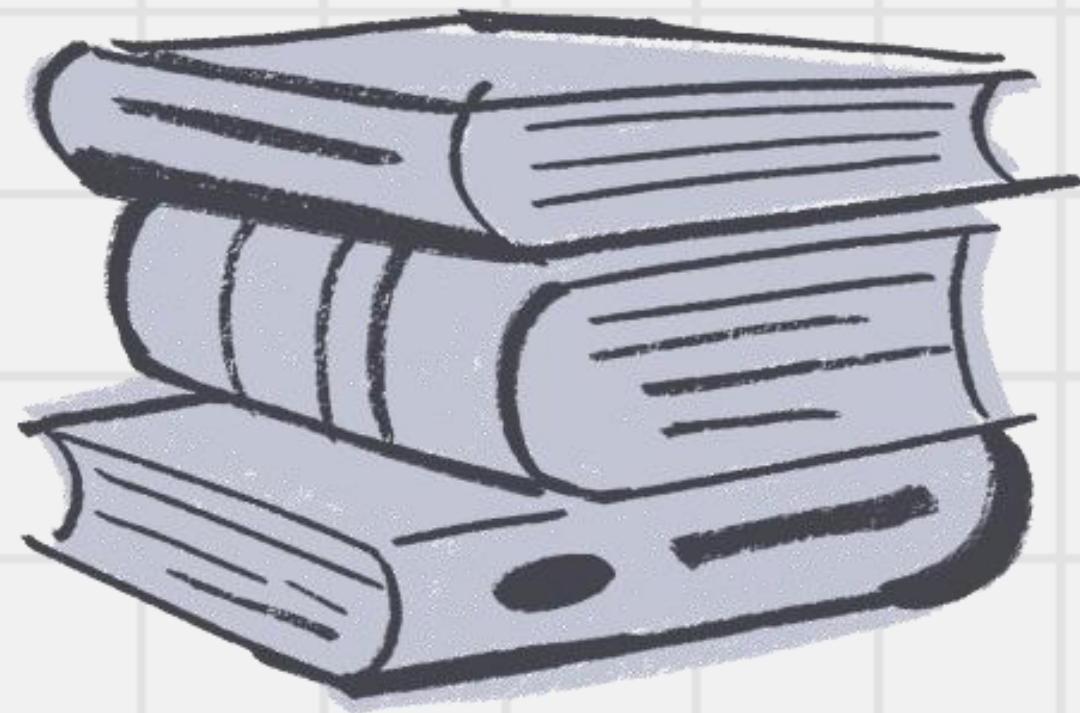




# COMPUTER VISION



# Table of Contents



1

Introduction

2

Real-time Camera

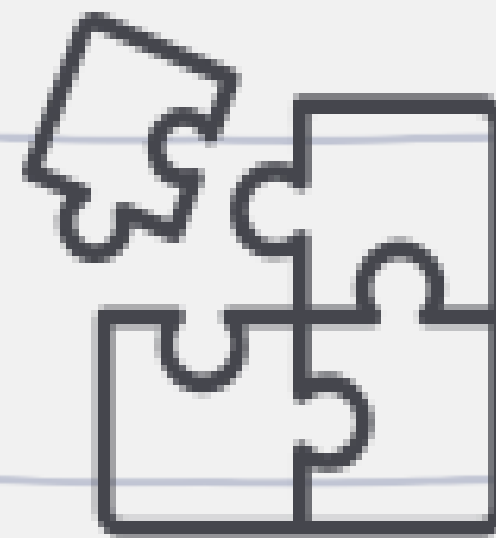
3

Wireless Real-time Cam





01



# Introduction

# Introduction

Now that we have most of the foundational knowledge, we're almost ready to build our project.

However, there are still a few technical steps we need to take.

The first of these is using a real-time camera to capture a live video stream, which is essential for performing object tracking.

In this session, we'll walk through that process step by step.



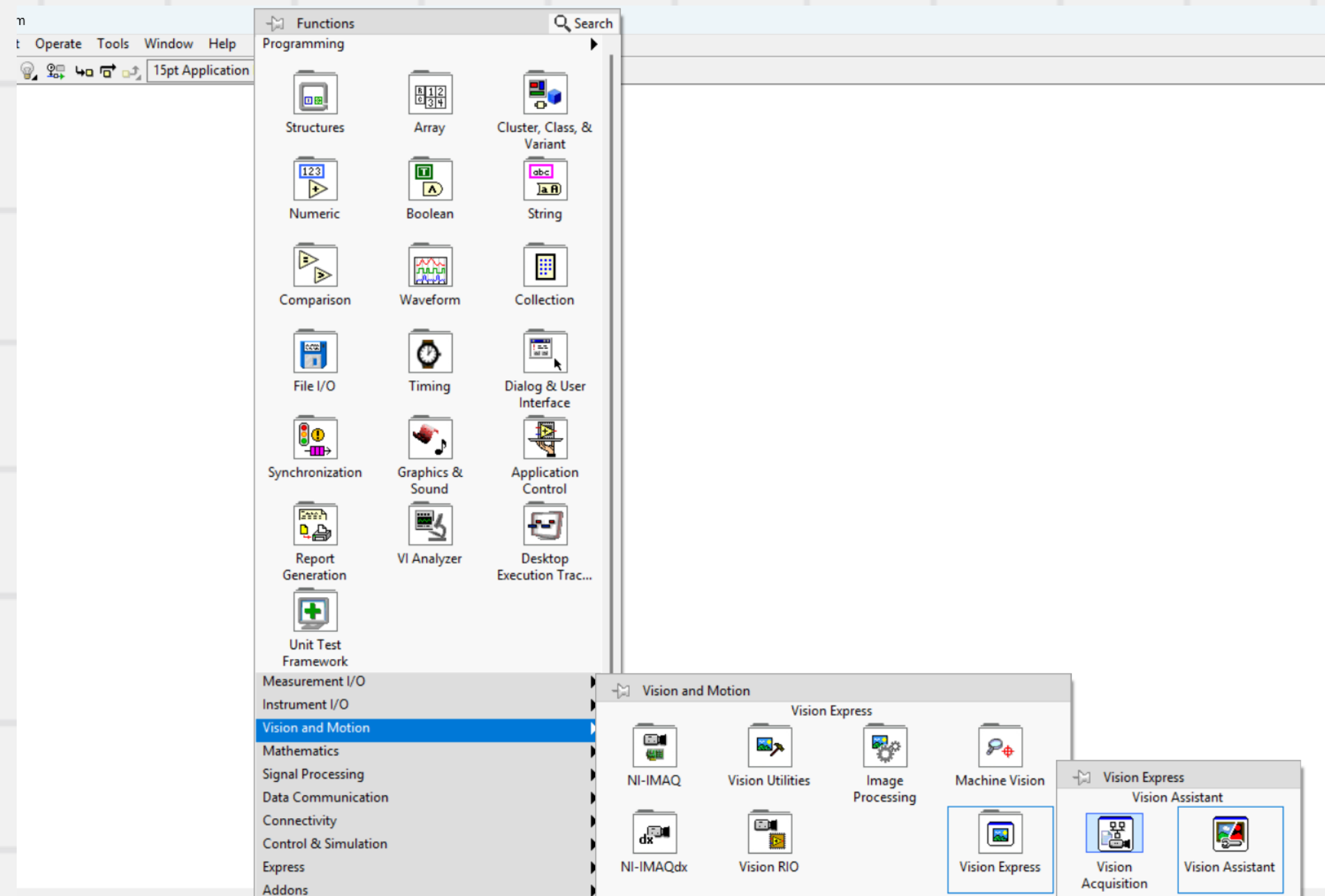
02



Real-time Camera

# Real-time Camera

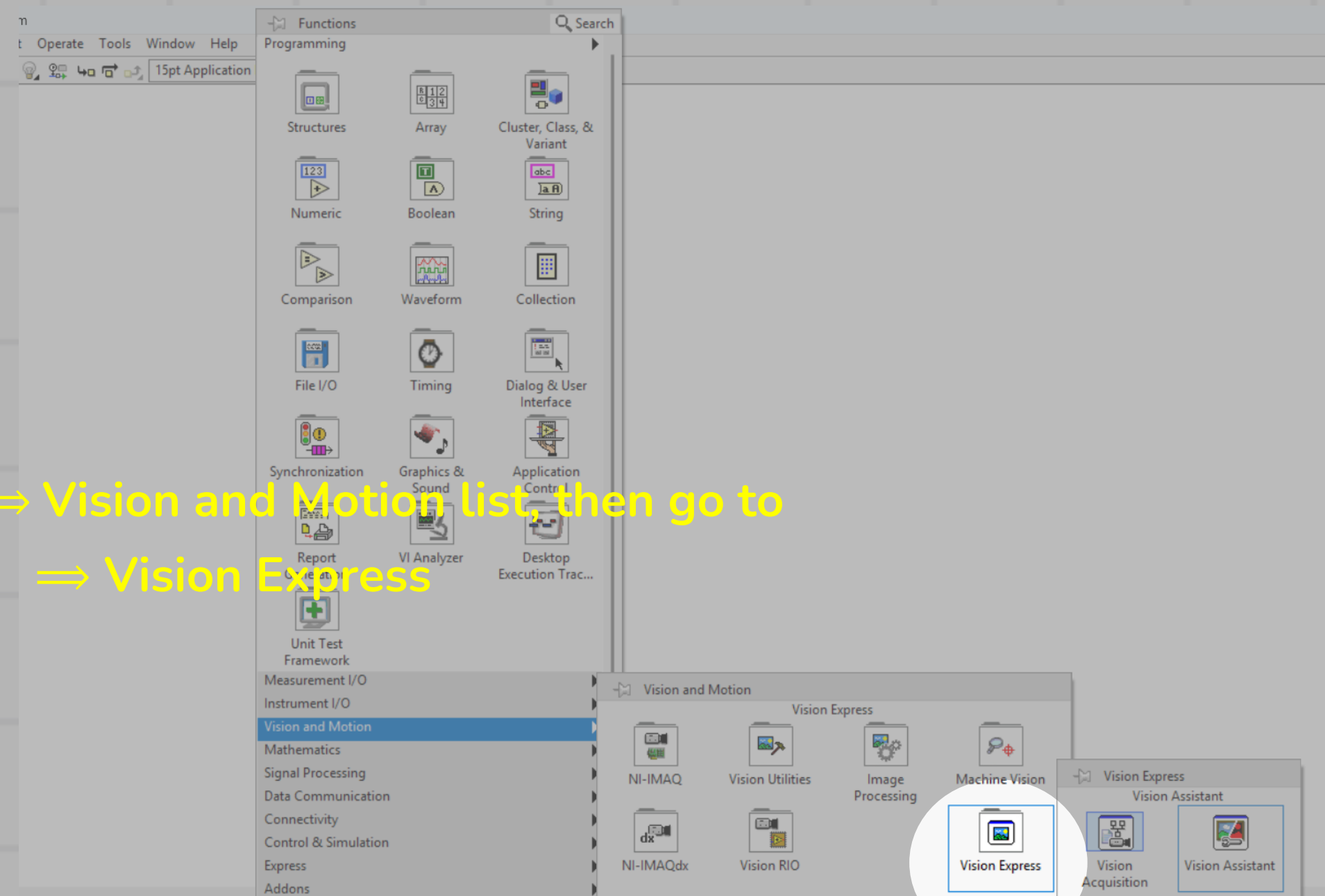
Simplest way to do this is by depending on vision acquisition app.



# Real-time Camera

Simplest way to do this is by depending on vision acquisition app.

You must go ⇒ Vision and Motion list, then go to  
⇒ Vision Express

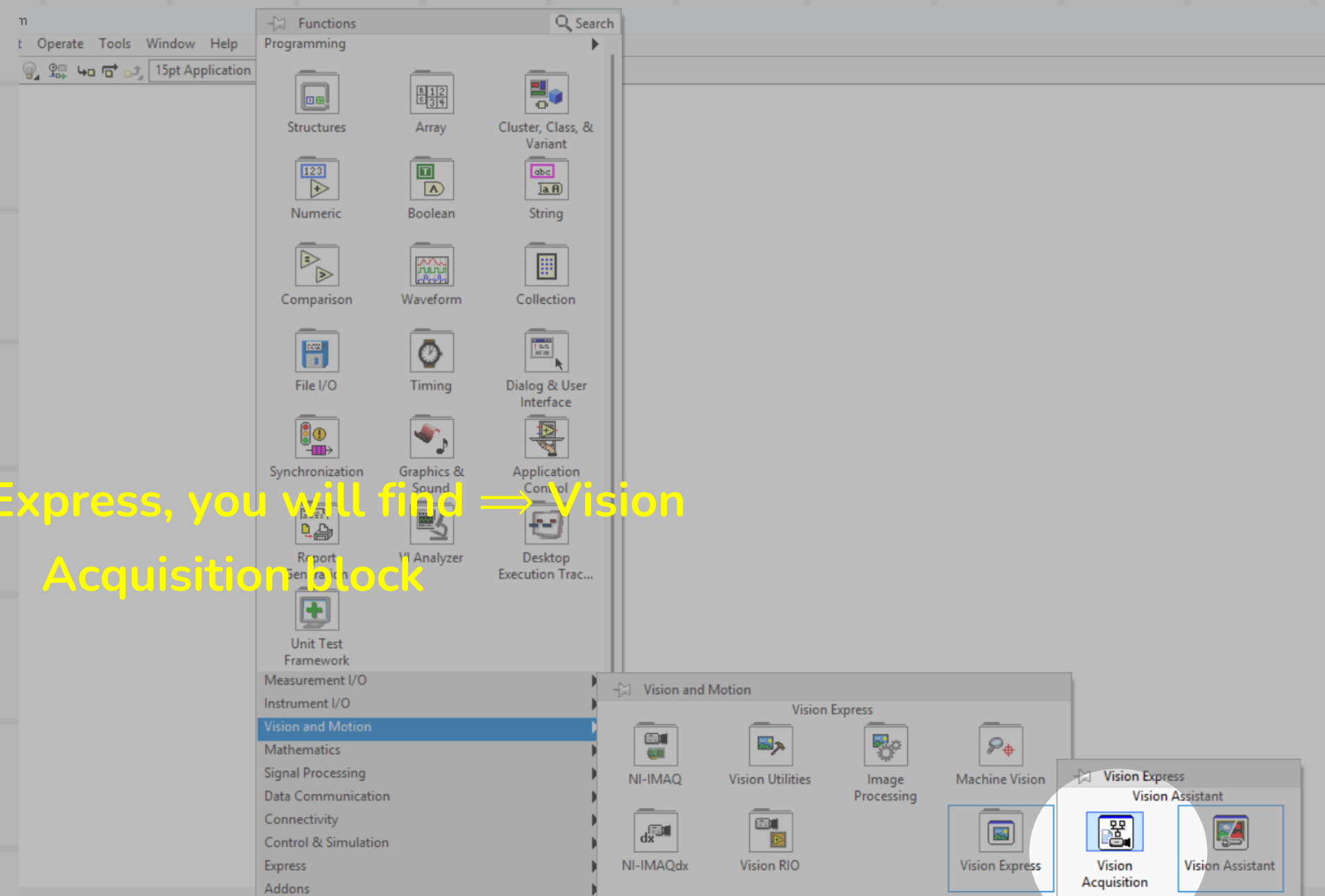




# Real-time Camera

Simplest way to do this is by depending on vision acquisition app.

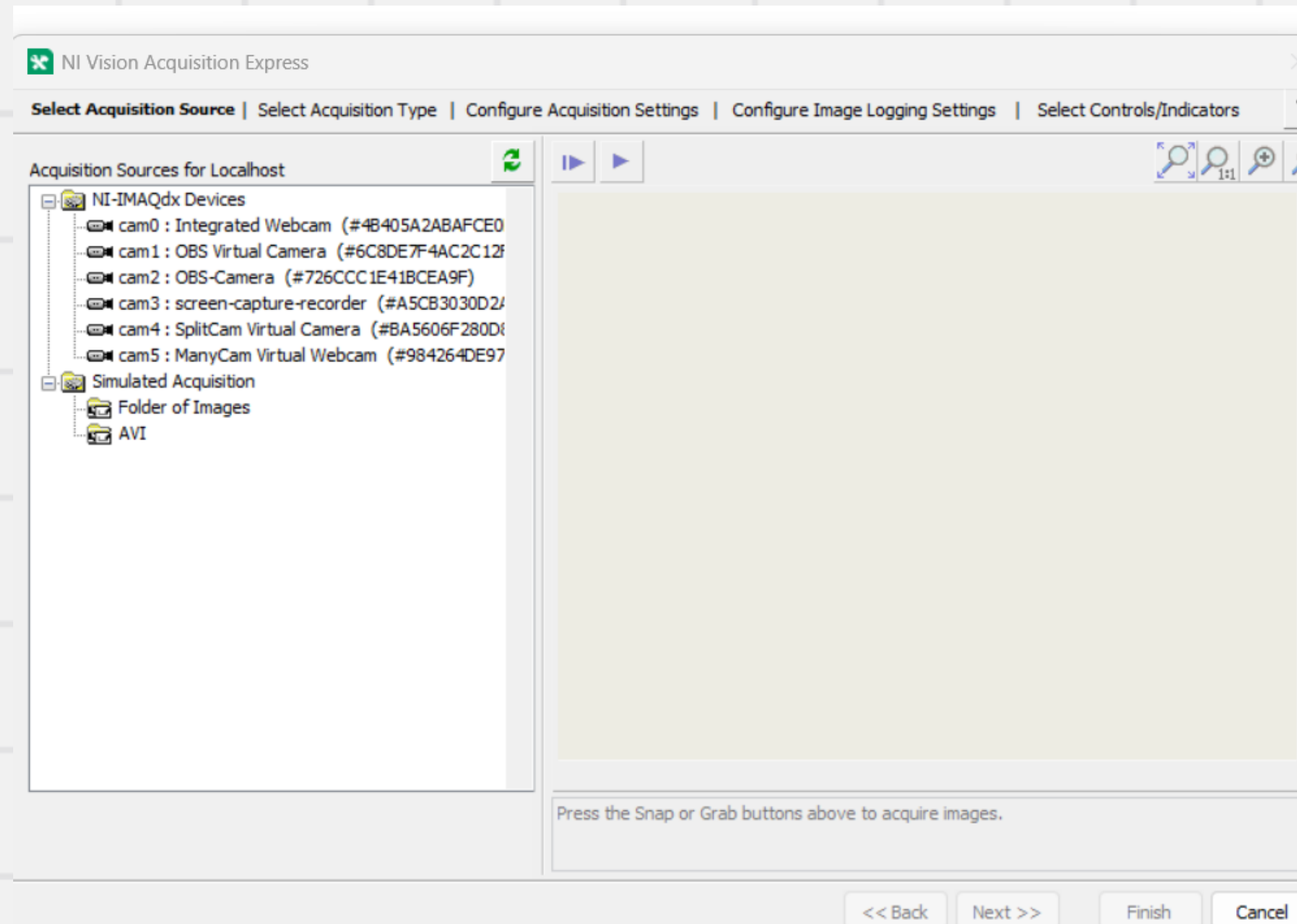
In Vision Express, you will find ⇒ Vision Acquisition Block





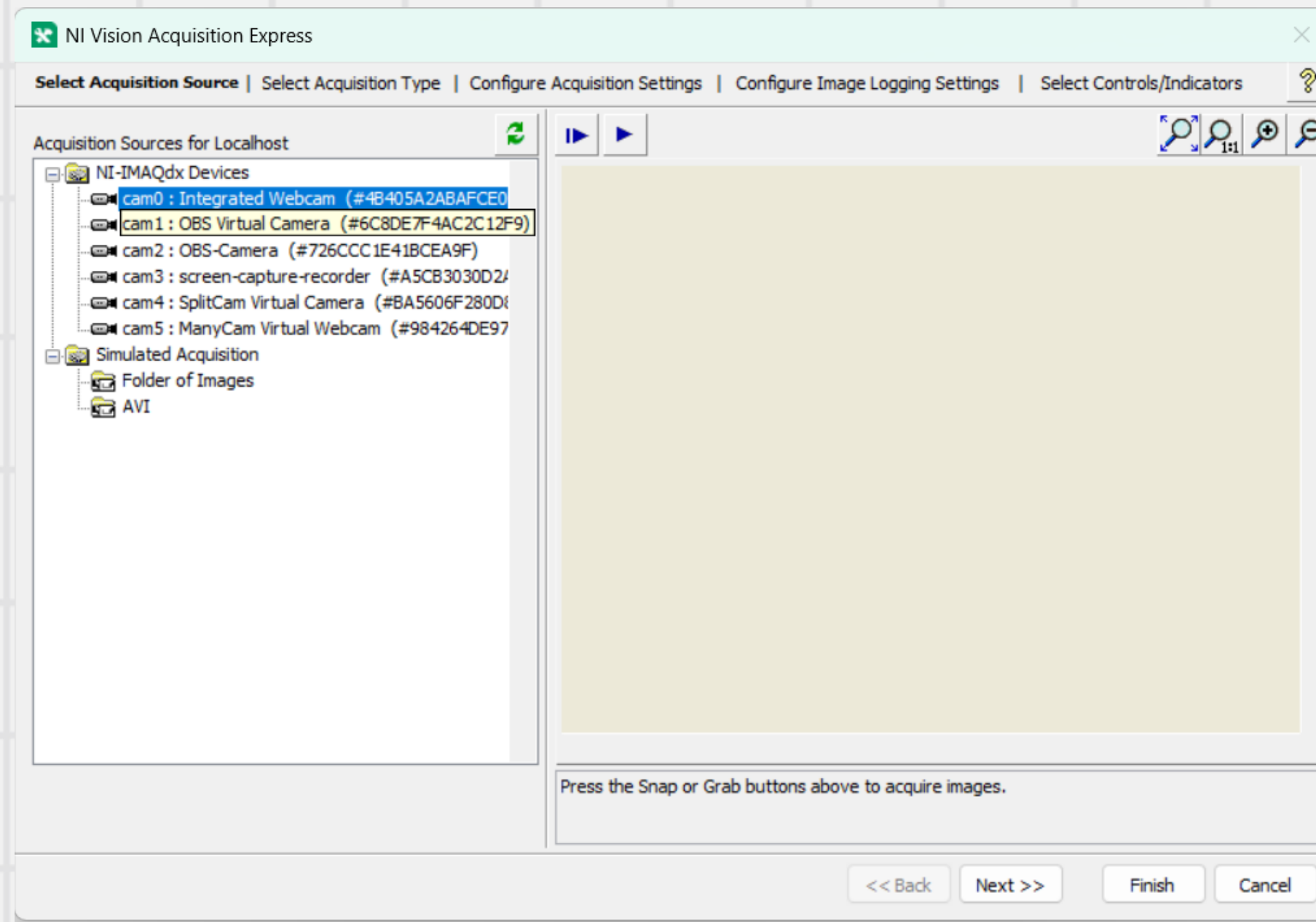
# Real-time Camera

When you drag and drop this block, A GUI window will open.

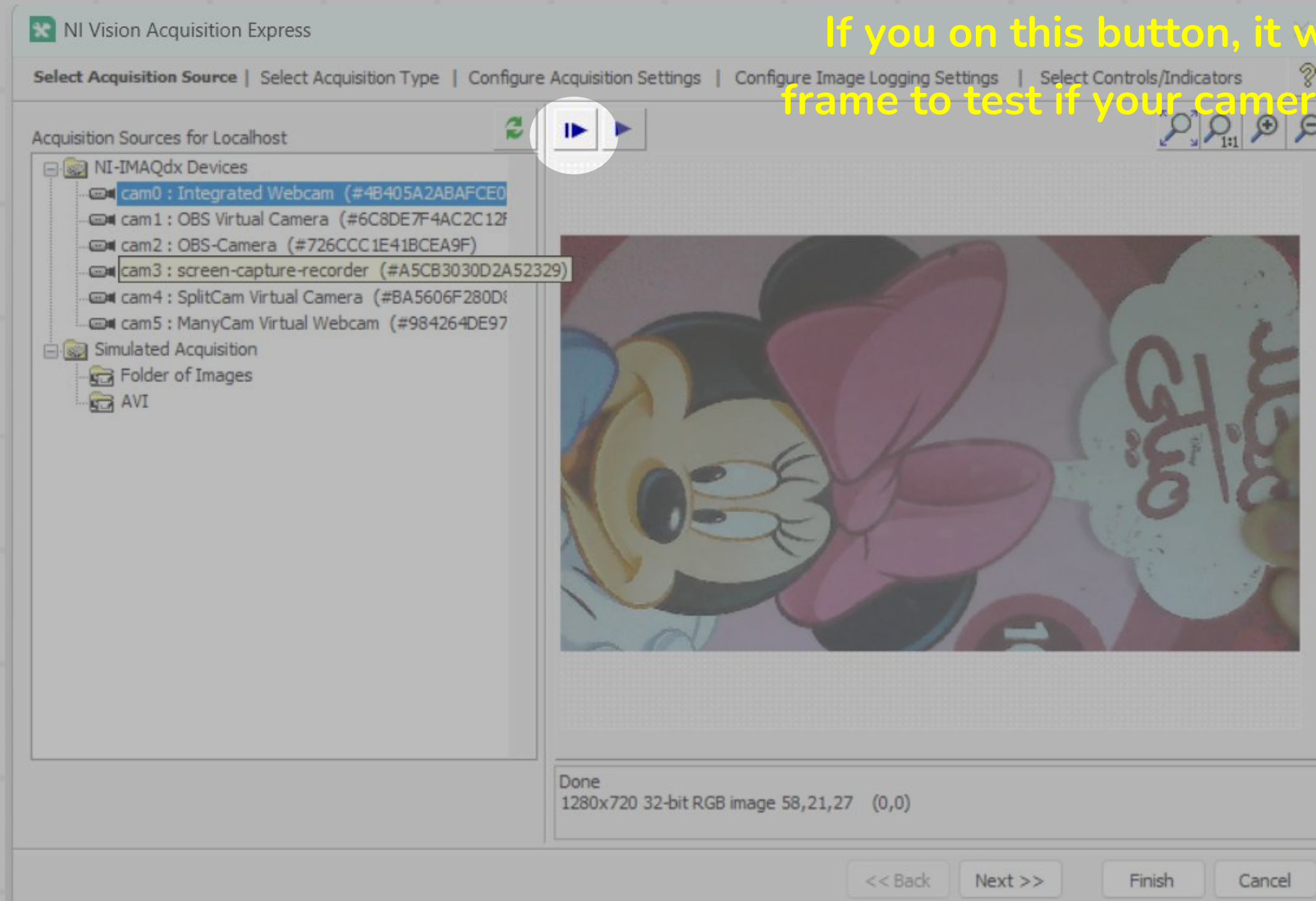


# Real-time Camera

Now, You will find your connected cameras and choose the needed one.

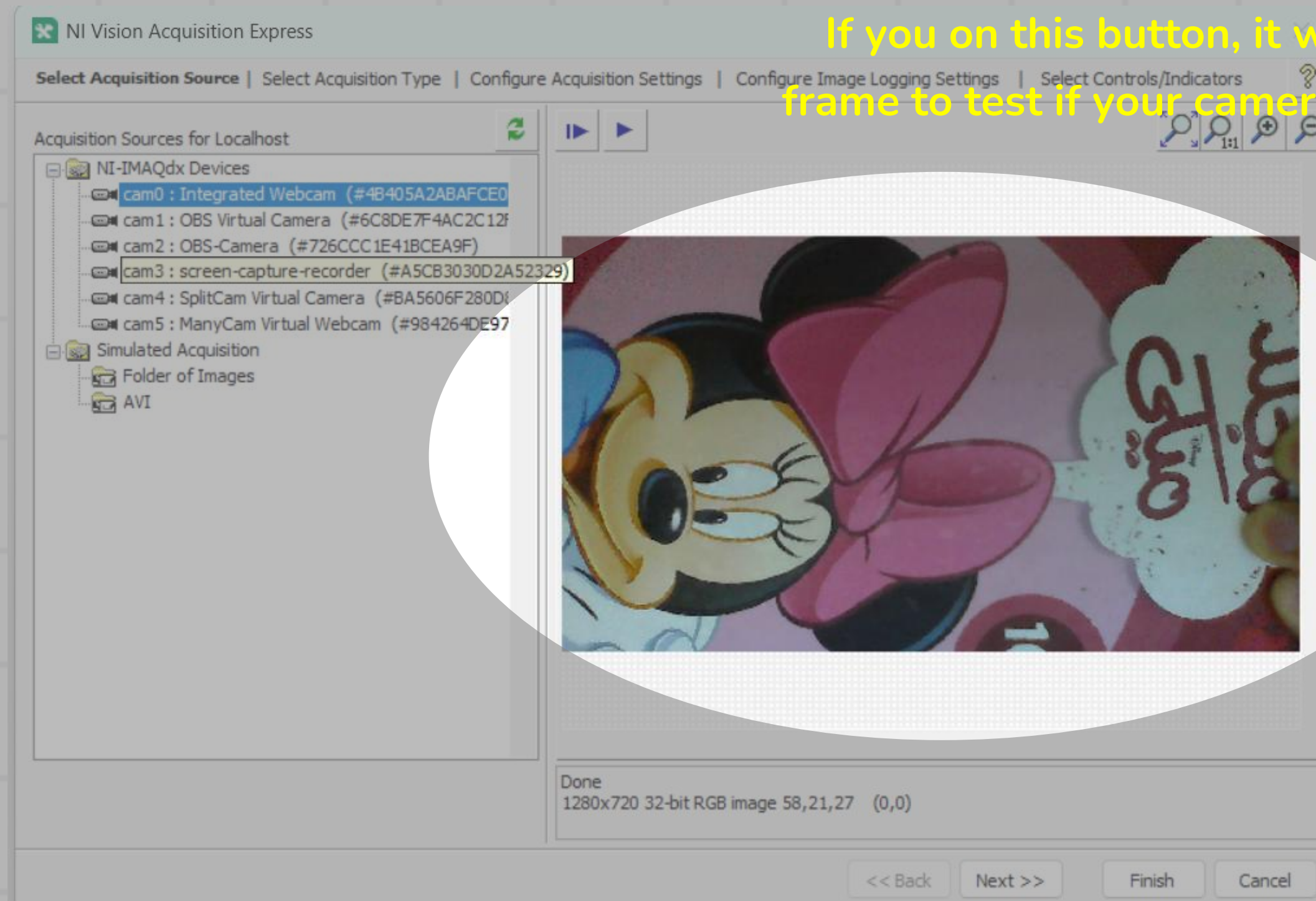


# Real-time Camera





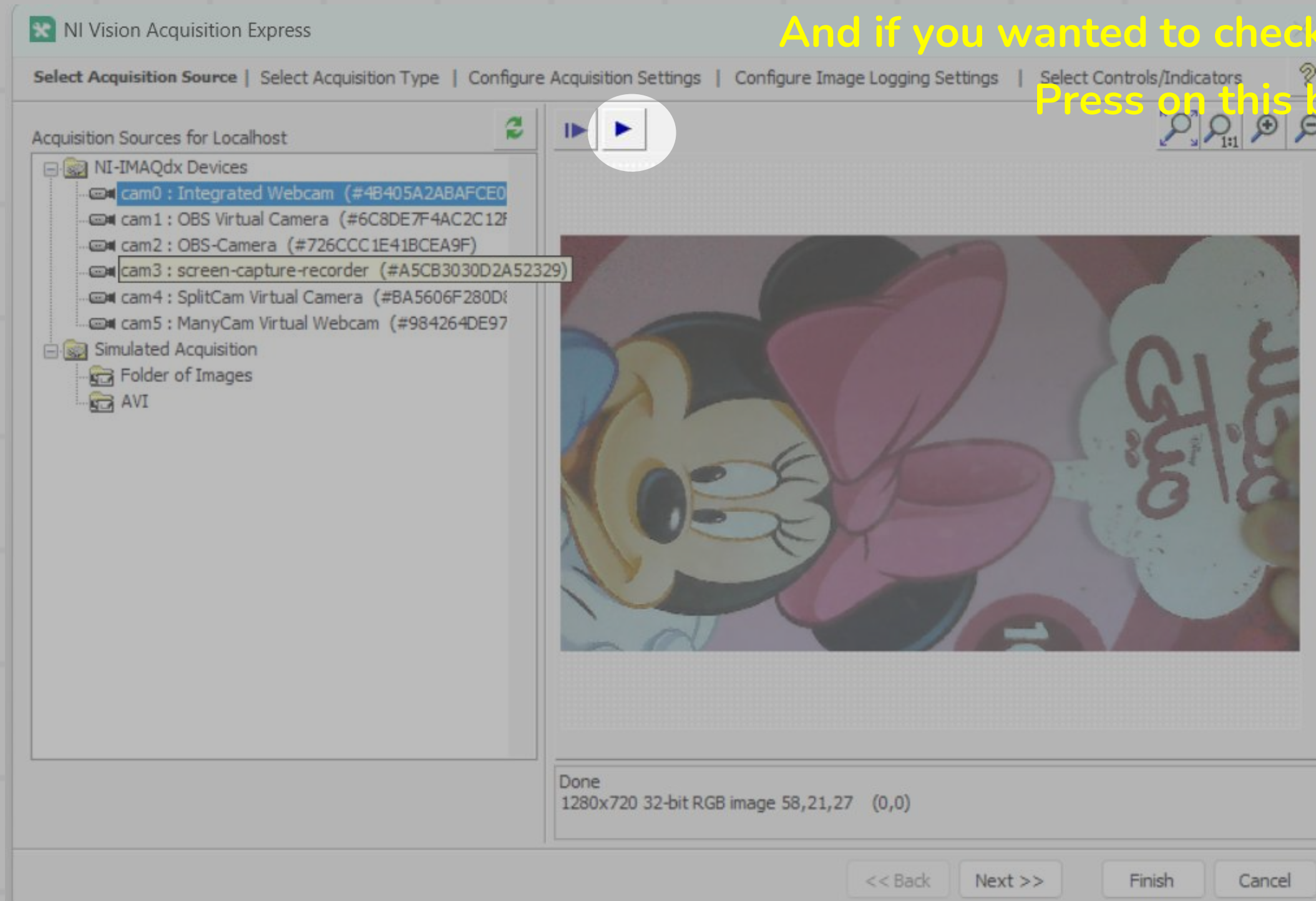
# Real-time Camera



If you on this button, it will snap only one frame to test if your camera is working or not.



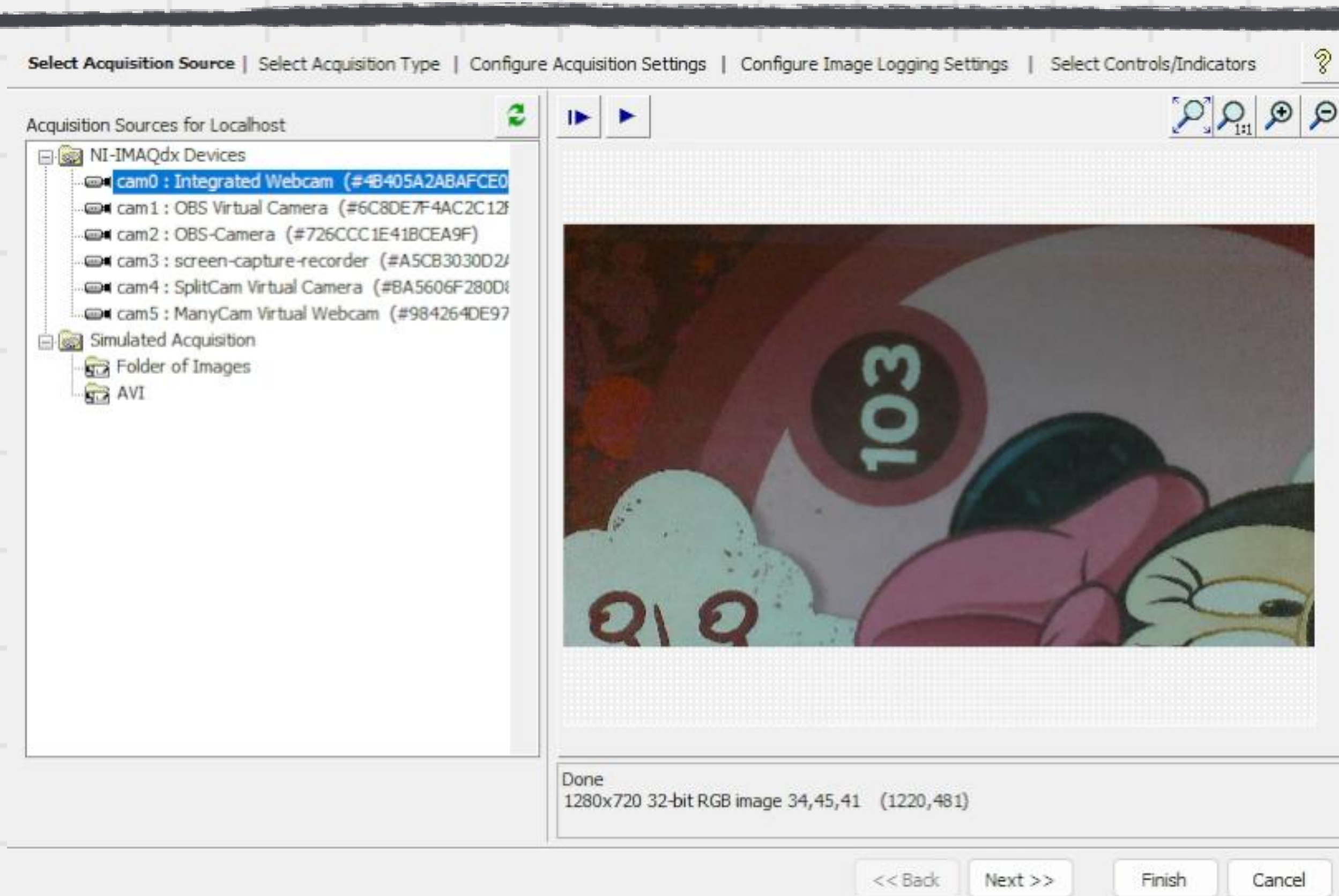
# Real-time Camera



And if you wanted to check the real-time cam.

Press on this button.

# Real-time Camera





# Real-time Camera

After pressing next, we will be able to select our suitable Acquisition type.

Select Acquisition Source | **Select Acquisition Type** | Configure Acquisition Settings | Configure Image Logging Settings | Select Controls/Indicators ?

---

☒ **Single Acquisition with processing**  
This acquisition is used for acquiring a single image. No loop structures are required.

☐ **Continuous Acquisition with inline processing**  
This acquisition is used for continuously acquiring images. If you do not want to miss images, select Acquire Every image and specify the Number of Images to buffer. Your average image processing time must be less than your image acquisition time to avoid missing images.

☐ **Finite Acquisition with inline processing**  
This acquisition is used for acquiring a fixed number images once. When an image is acquired, it will be available for image processing. This is useful if you want to display or process your images before the acquisition is done.

☐ **Finite Acquisition with post processing**  
This acquisition is used for acquiring a fixed number images once. The images will be available when all images have been acquired. This is useful if your image processing time is longer than your image acquisition time.

Acquire Image Type  
Acquire Most Recent Image

Number of Images to Buffer  
5

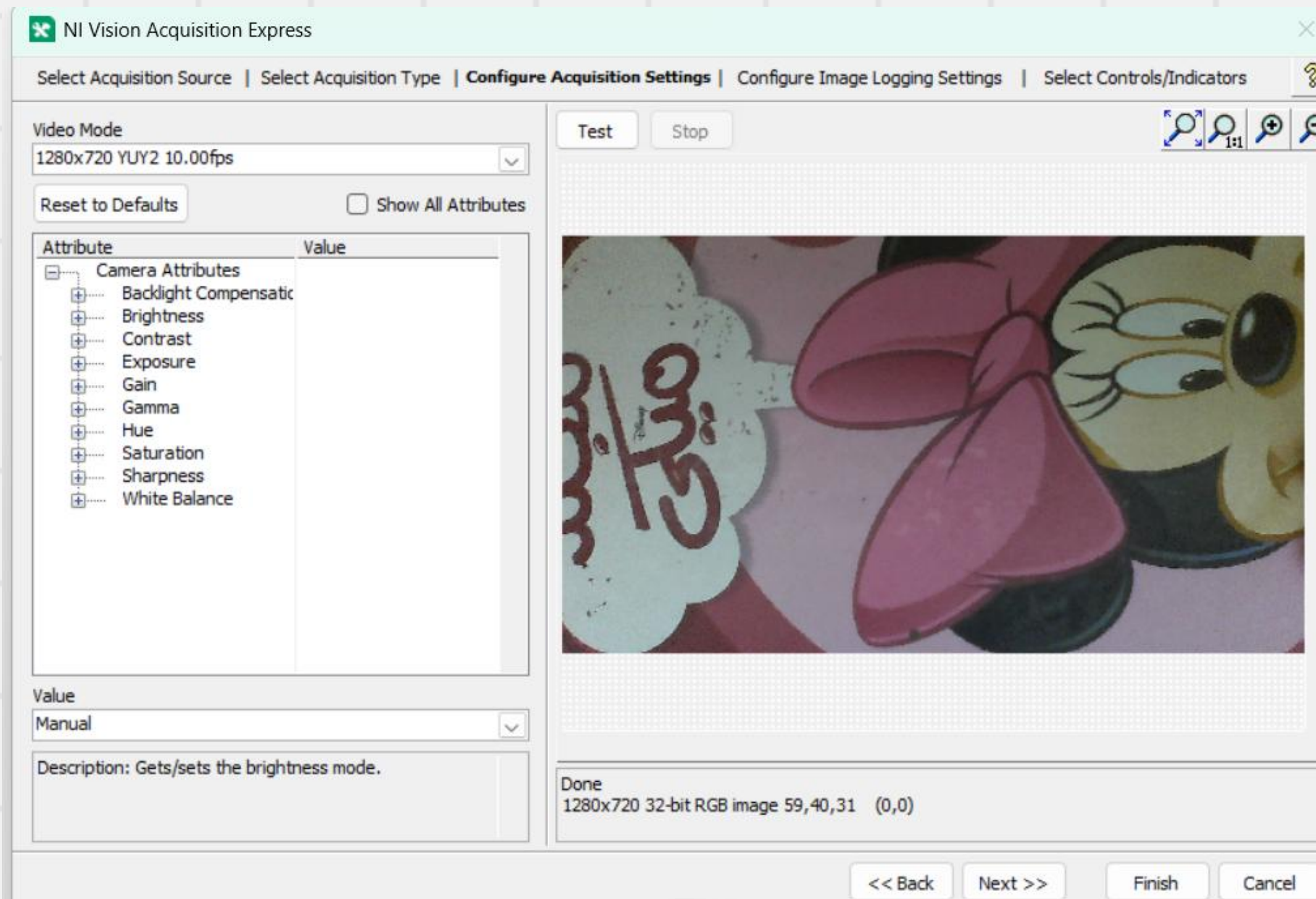
Number of Images to Acquire  
5

Number of Images to Acquire  
5

<< Back   Next >>   Finish   Cancel

# Real-time Camera

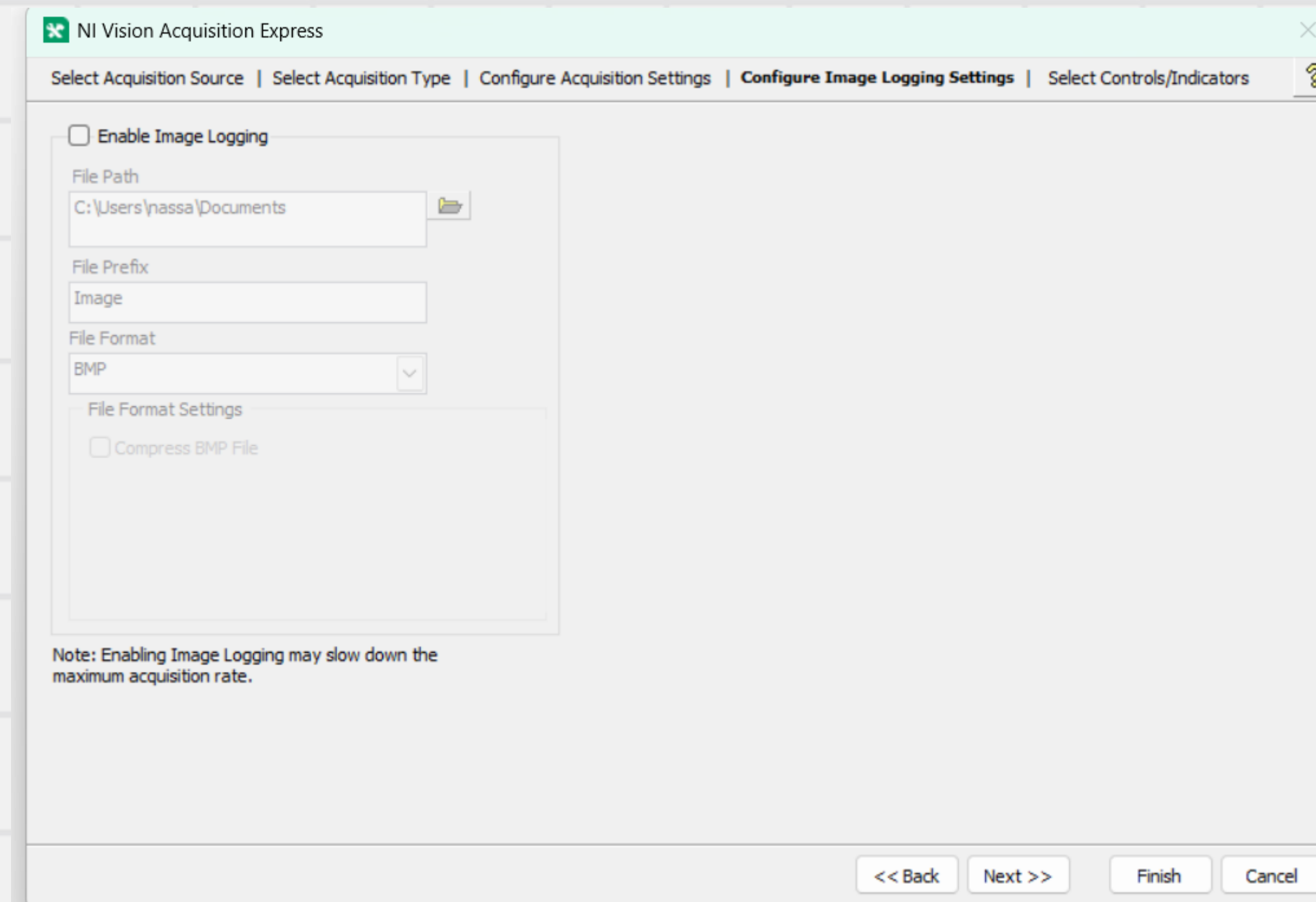
After that, we are going to choose the video mode which control the resolution and frame rate.





# Real-time Camera

Now, it will offer you to enable image logging. For me I will neglect this option.



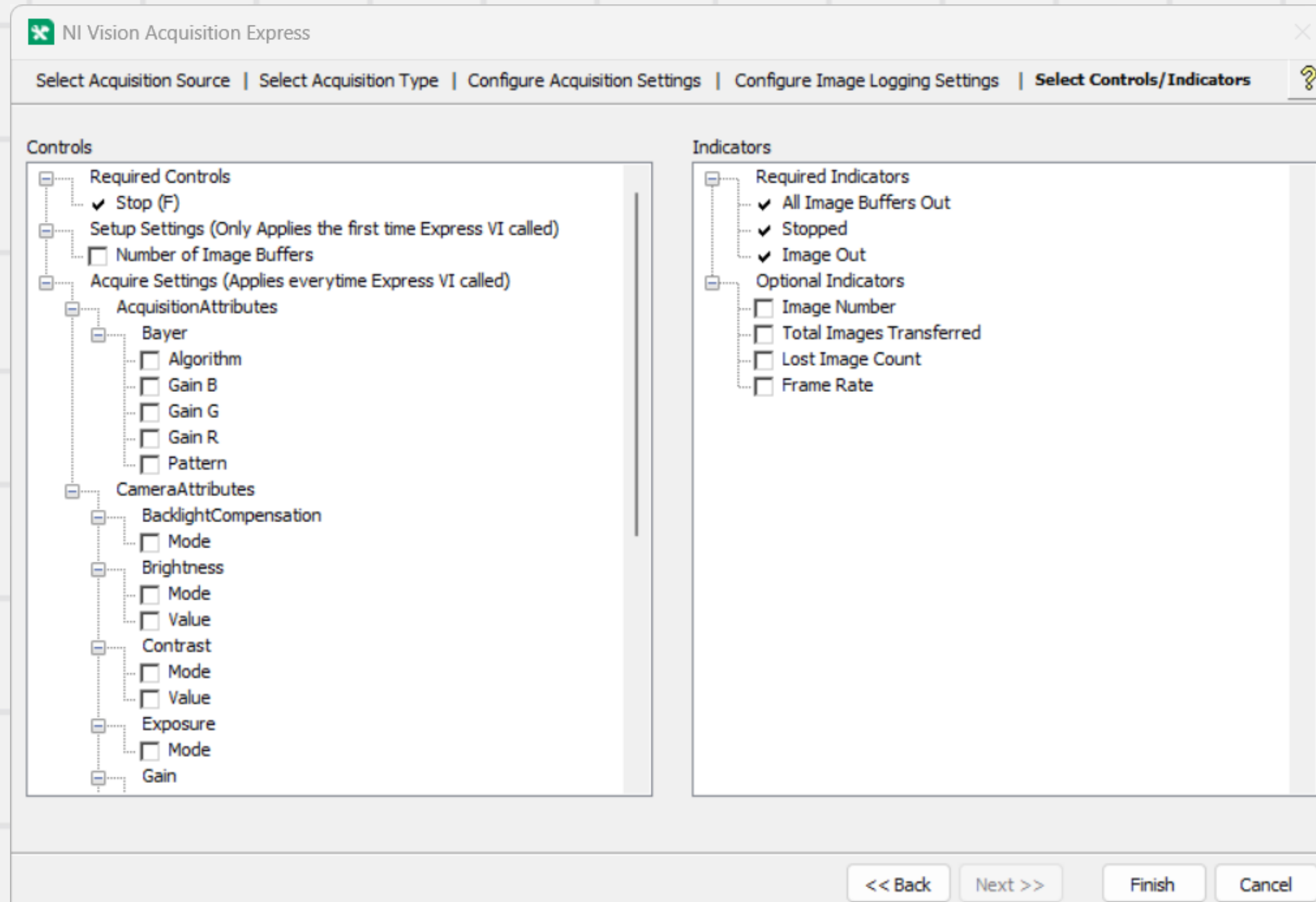
The screenshot shows the 'Configure Image Logging Settings' dialog box within the NI Vision Acquisition Express application. The dialog has a title bar with the application name and a close button. Below the title bar is a tabbed interface with five tabs: 'Select Acquisition Source', 'Select Acquisition Type', 'Configure Acquisition Settings', 'Configure Image Logging Settings' (which is the active tab), and 'Select Controls/Indicators'. The active tab contains the following settings:

- ☐ Enable Image Logging
- File Path: C:\Users\nassa\Documents (with a folder icon button)
- File Prefix: Image
- File Format: BMP (with a dropdown arrow)
- File Format Settings:
  - ☐ Compress BMP File

At the bottom of the dialog, there is a note: 'Note: Enabling Image Logging may slow down the maximum acquisition rate.' The bottom of the dialog features four buttons: '<< Back', 'Next >>', 'Finish', and 'Cancel'.

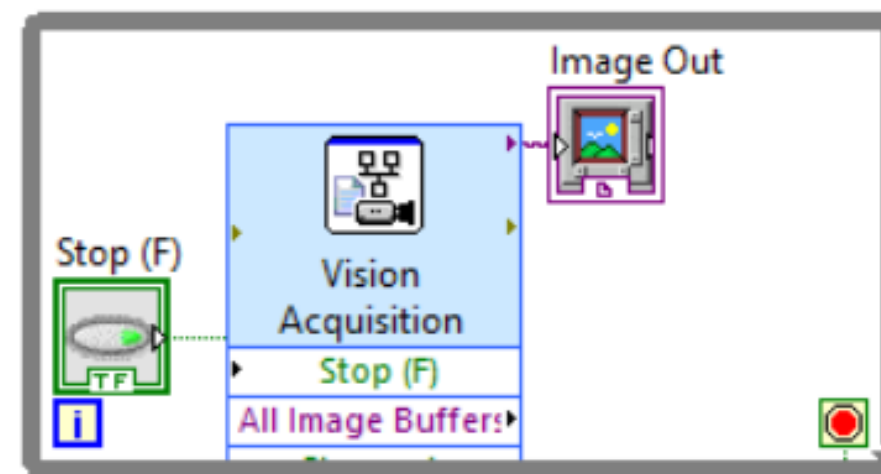
# Real-time Camera

Finally, you will be able to enable controls and indicators.



# Real-time Camera

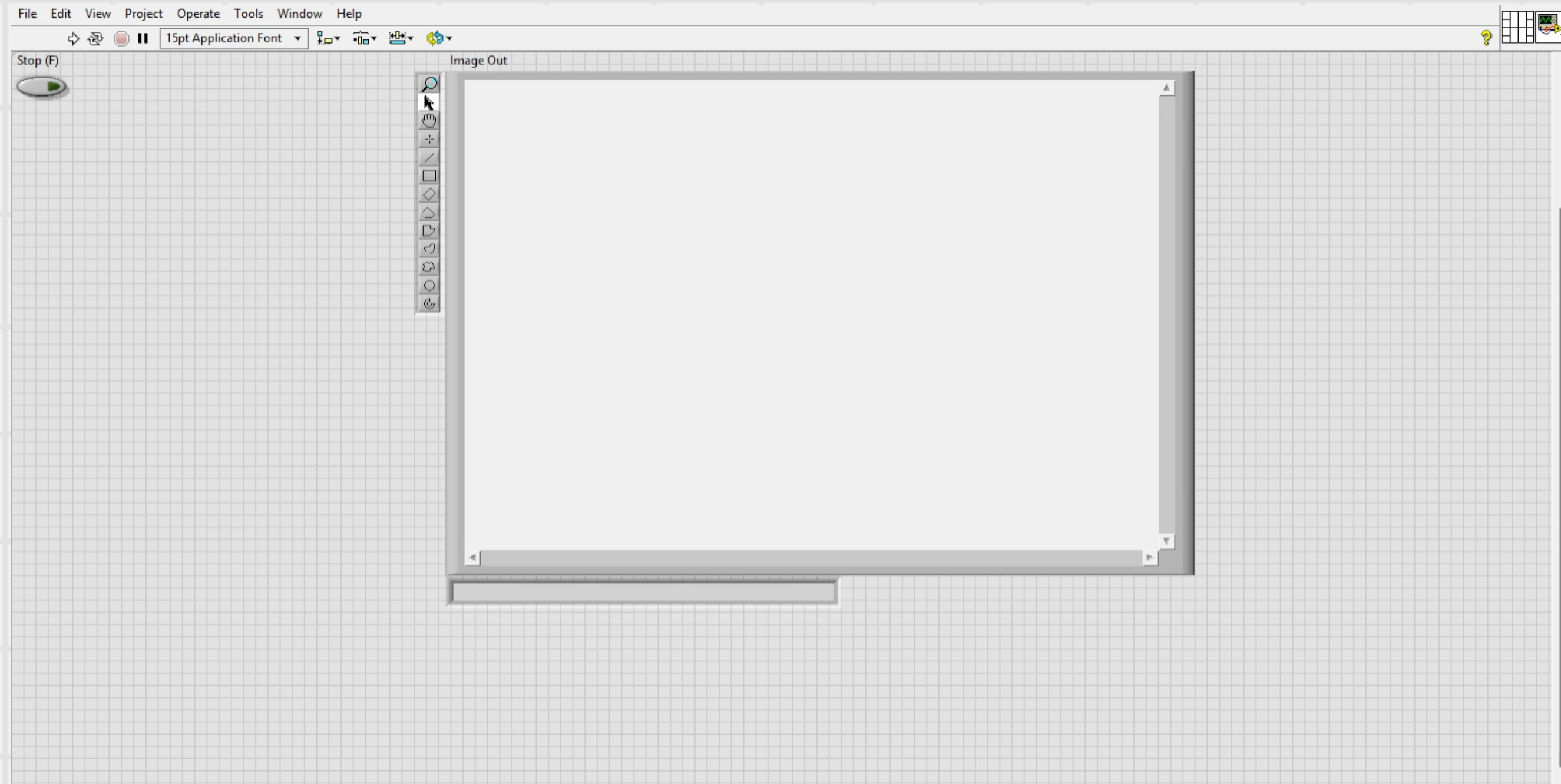
After pressing on finish, You will find that a while loop was generated in block diagram





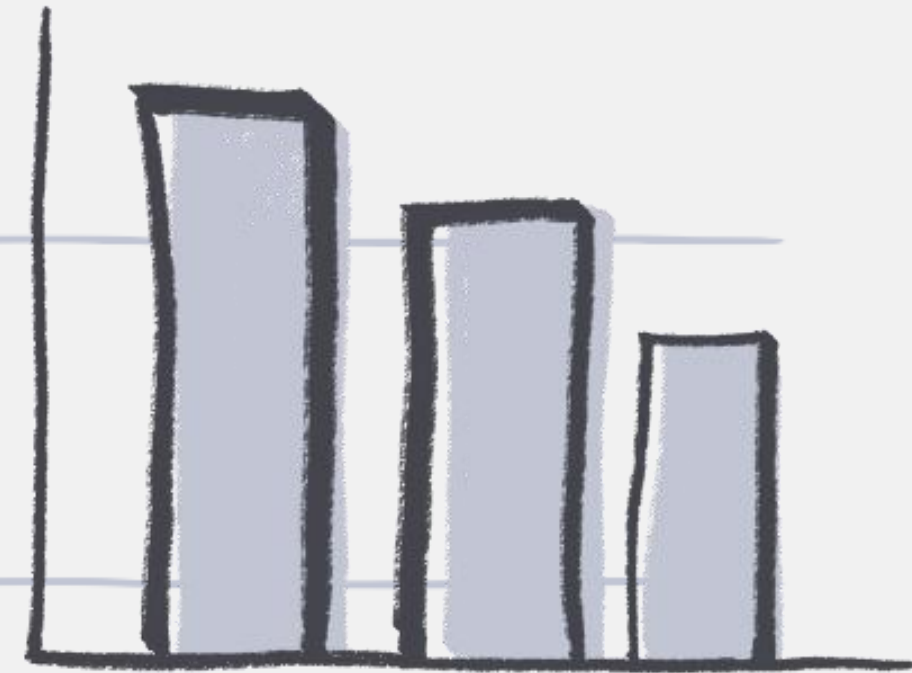
# Real-time Camera

If you went to front panel, you will find Image out window.





03



Wireless Real-time Cam

# Wireless Real-time Cam

In robotics and autonomous systems, accessing a real-time camera feed is essential for tasks like object tracking, navigation, and environment perception. While LabVIEW offers built-in support for USB and wired cameras, it does not directly support wireless IP cameras out of the box.

In this chapter, we'll explore how to overcome this limitation by integrating a wireless real-time camera—such as an IP camera—into LabVIEW. This allows us to stream video wirelessly, enabling more flexible and mobile robotic applications. Step by step, we'll demonstrate how to receive the video stream, process it within LabVIEW, and use it as a foundation for computer vision tasks in robotics.



# Wireless Real-time Cam



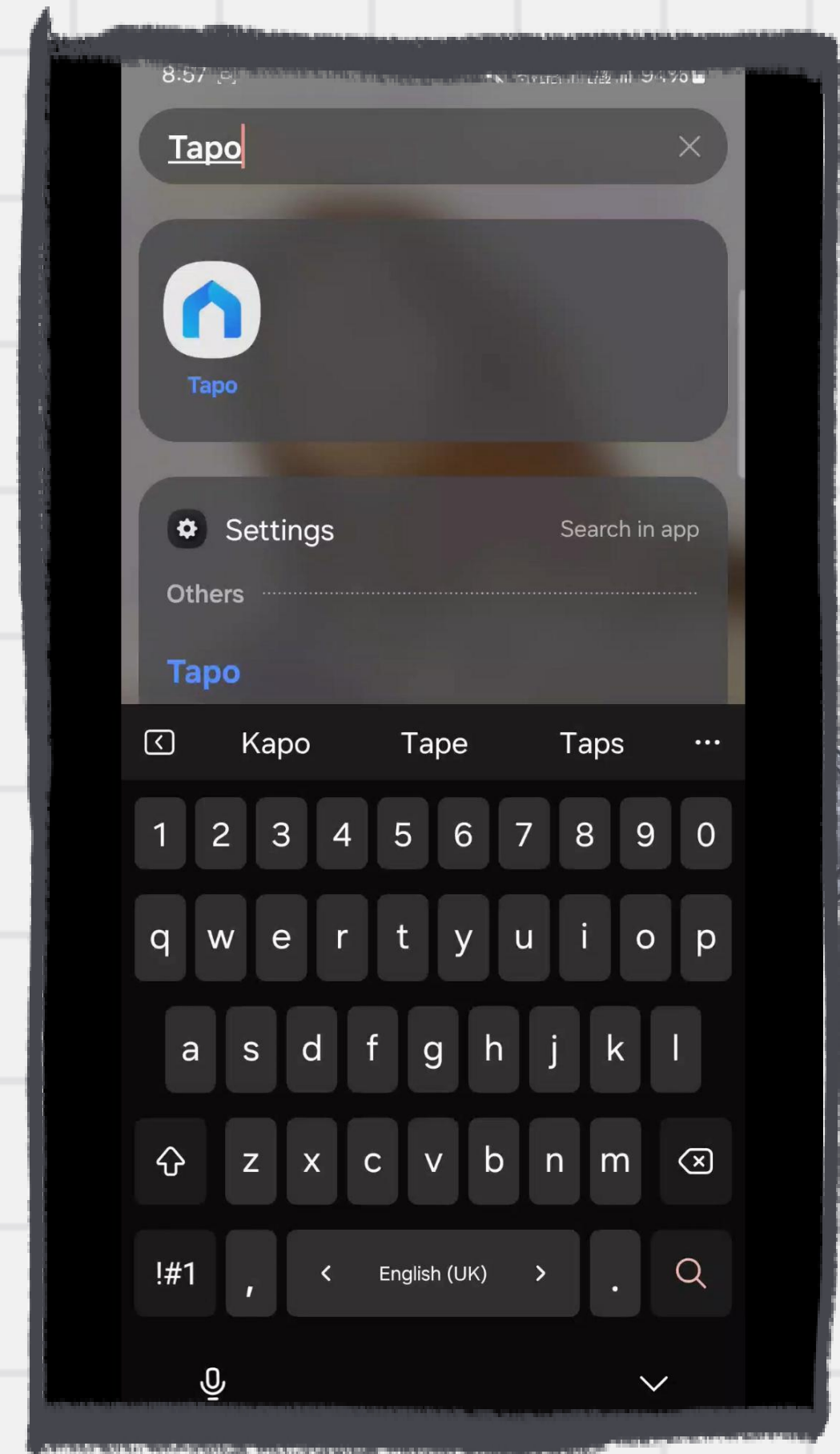
For this project, we use the TP-Link Tapo C100, a budget-friendly wireless IP camera that offers reliable video streaming over Wi-Fi. It supports up to 1080p resolution, night vision, and motion detection, making it a great choice for basic surveillance and computer vision applications.

While the Tapo C100 is not natively supported by LabVIEW, we can access its RTSP (Real-Time Streaming Protocol) video stream and integrate it into our LabVIEW environment using external tools. This allows us to use it as a wireless real-time camera, ideal for robotics and autonomous systems where mobility and flexibility are essential.

In this session, we'll show how to connect the Tapo C100 to LabVIEW and stream live video for further processing and object tracking.

# Wireless Real-time Cam

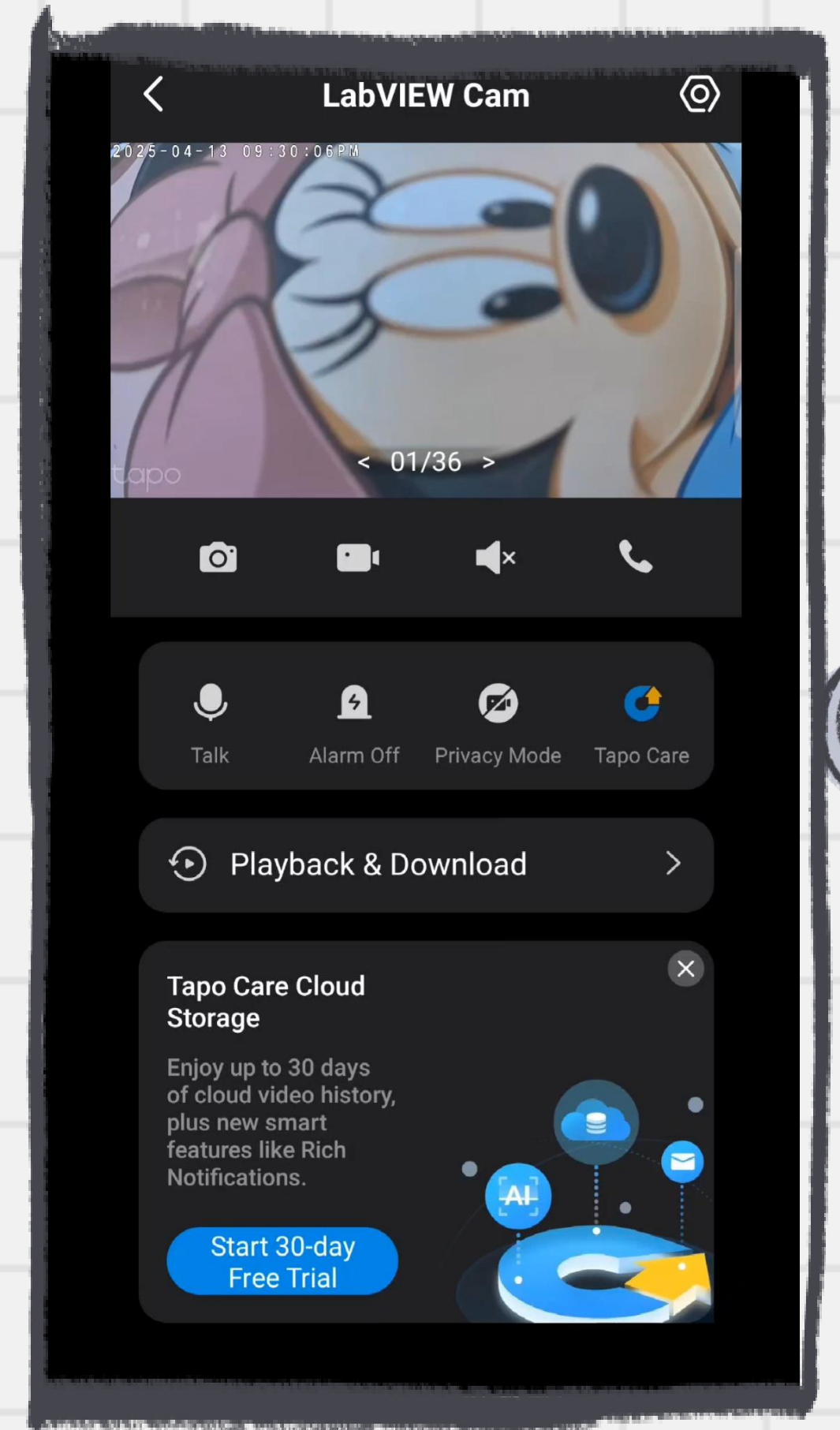
In this video, you will see how to start your camera's stream.



# Wireless Real-time Cam



Now, we are ready to stream Tapo c100 video to our host machine.

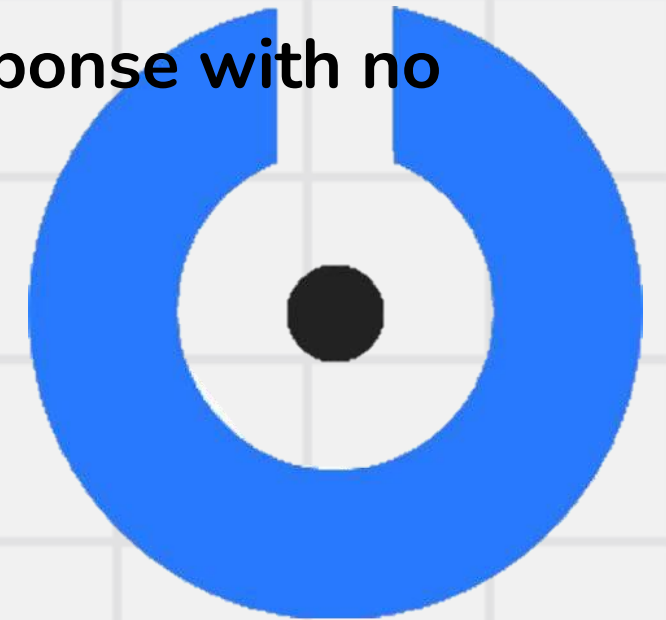




# Wireless Real-time Cam



Now, we will install app which can monitor the stream using the IP.  
I tried many apps, and I found that SplitCam has the fastest response with no delay.



# Wireless Real-time Cam

Add the following address to  
you IP camera input

`"rtsp://$USER_NAME$: $PASSWORD$@192.168.xx.xx:554/stream1"`

# Wireless Real-time Cam

Now that the video stream is being published to our host machine, we face another challenge:  
how to use this stream as an input to LabVIEW.

As mentioned earlier, LabVIEW only supports image acquisition from wired cameras through drivers like Vision Acquisition. This means it doesn't natively accept video streams from wireless IP cameras.

So, how can we make this stream act like a wired camera?

The answer is simple: we use tools that create a virtual camera. A virtual camera appears to the system—and to LabVIEW—as if it were a real, wired webcam. However, its source can actually be an RTSP stream, a screen capture, or any other video input.

we'll show how to use virtual camera software to bridge the gap between the wireless Tapo C100 and LabVIEW, enabling real-time video processing for our robotics applications.

# Wireless Real-time Cam

SplitCam offers virtual cam but for some reason it doesn't work with LabVIEW, but I found app called VCam which worked seamlessly with LabVIEW. So, I take the SplitCam virtual cam as input in VCam, then I use VCam virtual cam as input at LabVIEW.



VCam

My Workspace ▾



Background



Logo



Camera



Name Tag



Help



Feedback



Settings

## Background



Original



Remove



Blur



+ Add Background

⚡ Upgrade

2025-04-14 03:59:09 AM



tapo



VCam.ai



Resume Camera



THANK

you